UNIVERSITÉ DE YAOUNDÉ I

Faculté des Sciences

Département d'Informatique

Rapport d'Implémentation

API de Transaction Bancaire

Devoir 304 — Conception d'API REST

Étudiant : KOA ESSAMA PAULIN BRICE

Matricule : 25I2255

Niveau : Licence 3 ICT

Établissement : Université de Yaoundé I

Dépôt GitHub : https://github.com/eliote-geeks/Devoir_304

Date : Avril 2026

« Conception et implémentation d'une API REST pour la gestion de comptes bancaires, de dépôts et de retraits. »

0 Table des matières

1 Introduction et contexte **3**

1.1 Contexte du projet . . . 3

1.2 Problème résolu . . . 3

1.3 Choix technologiques . . . 3

2 Architecture du projet **3**

2.1 Structure des fichiers . . . 3

2.2 Architecture en couches . . . 4

3 Spécifications fonctionnelles **4**

3.1 Acteurs du système . . . 4

3.2 Les 5 fonctions du système . . . 5

3.3 Règles de gestion . . . 5

4 Spécifications non fonctionnelles **5**

5 Implémentation détaillée — fichier par fichier **5**

5.1 `models.py` — Définition des structures de données . . . 6

5.1.1 AccountCreate — Données pour créer un compte . . . 6

5.1.2 TransactionRequest — Données pour un dépôt ou retrait . . . 6

5.1.3 Autres modèles — Réponses de l'API . . . 7

5.2 `storage.py` — Persistance des données . . . 7

5.2.1 Configuration et utilitaires . . . 7

5.2.2 Initialisation et lecture/écriture . . . 8

5.2.3 Création d'un compte . . . 9

5.2.4 Application d'une transaction (dépôt ou retrait) . . . 10

5.3 `main.py` — Les routes de l'API . . . 11

5.3.1 Initialisation de FastAPI . . . 11

5.3.2 Fonction 1 — Créer un compte . . . 11

5.3.3 Fonction 2 — Lister les comptes . . . 12

5.3.4 Fonction 3 — Consulter un compte . . . 12

5.3.5 Fonction 4 — Effectuer un dépôt . . . 13

5.3.6 Fonction 5 — Effectuer un retrait . . . 13

6 Cas de test **14**
6.1 Tableau complet des 23 cas de test 14
6.2 Test manuel détaillé — Scénario complet 15
6.2.1 Étape 1 — Vérifier la liste vide (CT-06) 15
6.2.2 Étape 2 — Créer un compte (CT-01) 15
6.2.3 Étape 3 — Tester la validation (CT-03, CT-04, CT-05) 16
6.2.4 Étape 4 — Effectuer un dépôt (CT-10) 16
6.2.5 Étape 5 — Effectuer un retrait valide (CT-15) 17
6.2.6 Étape 6 — Tester le solde insuffisant (CT-17) 17
6.2.7 Étape 7 — Consulter l'historique (CT-22) 17
6.2.8 Étape 8 — Tester un ID inexistant (CT-09) 18

7 Documentation Swagger **18**
7.1 Accès à l'interface 18
7.2 Exemples interactifs dans Swagger 18
7.3 Comment utiliser Swagger pour le test manuel 18

8 Lancement et déploiement **19**
8.1 Lancement en local 19
8.2 Contenu de requirements.txt 19
8.3 Déploiement sur Render 19

9 Conclusion **19**

1 Introduction et contexte

1.1 Contexte du projet

Ce rapport décrit de façon complète et détaillée l'implémentation d'une **API REST de transaction bancaire** réalisée dans le cadre du Devoir 304 de Licence 3 ICT.

L'objectif est de concevoir un système permettant à des clients de gérer leurs comptes bancaires numériquement : créer un compte, consulter son solde, effectuer des dépôts et des retraits, et visualiser l'historique de leurs opérations.

1.2 Problème résolu

Les établissements financiers ont besoin de solutions numériques pour gérer les opérations de base. Cette API constitue le back-end (côté serveur) d'un tel système, pouvant être consommée par une application web, mobile ou desktop.

1.3 Choix technologiques

Composant	Technologie	Raison du choix
Langage	Python 3.13	Lisible, productif, écosystème riche
Framework	FastAPI	Moderne, rapide, génère Swagger automatiquement
Serveur	Uvicorn	Serveur ASGI haute performance
Persistance	Fichier JSON local	Simple, portable, sans dépendance externe
Documentation	Swagger UI (OpenAPI 3.0)	Standard industriel, interactif
Validation	Pydantic v2	Validation automatique des données entrantes

2 Architecture du projet

2.1 Structure des fichiers

```
Devoir_304/
|
|-- app/                        # Package principal de l'API
|    |-- __init__.py            # Marque app/ comme package Python
|    |-- main.py                # Serveur FastAPI : routes et logique
    HTTP
|    |-- models.py              # Schemas de validation (Pydantic)
|    |-- storage.py             # Persistance : lecture/ecriture JSON
|    |-- static/                # Fichiers statiques (Swagger UI)
|         |-- swagger-ui/
|              |-- swagger-ui-bundle.js
|              |-- swagger-ui.css
|              |-- favicon.svg
|
|-- data/
```

```
|    |-- accounts.json          # Base de donnees locale (fichier JSON)
|
|-- requirements.txt            # Dependances Python
|-- render.yaml                 # Configuration deploiement Render
|-- CAHIER_DE_CHARGES.md        # Cahier des charges complet
```

Listing 1 – Arborescence du projet

2.2 Architecture en couches

L'API est organisée en **3 couches** distinctes, selon le principe de séparation des responsabilités :

Couche	Fichier	Rôle
Présentation	`main.py`	Définit les routes HTTP, reçoit les requêtes, retourne les réponses JSON
Modèle	`models.py`	Définit les structures de données et règles de validation
Données	`storage.py`	Gère la lecture et l'écriture dans le fichier `accounts.json`

> **Pourquoi cette séparation ?** Elle permet de modifier chaque couche indépendamment. Par exemple, on pourrait remplacer le fichier JSON par une vraie base de données (PostgreSQL) en modifiant uniquement `storage.py`, sans toucher aux routes ni aux modèles.

3 Spécifications fonctionnelles

3.1 Acteurs du système

Client bancaire : l'utilisateur final qui interagit avec le système. Il peut :

- ouvrir un compte bancaire
- consulter les informations de son compte
- effectuer des dépôts
- effectuer des retraits
- consulter son historique de transactions

Administrateur système : supervise le système. Il peut :

- consulter la liste complète des comptes
- suivre les transactions

3.2 Les 5 fonctions du système

#	Fonction	Méthode HTTP	URL
F1	Créer un compte	POST	`/accounts`
F2	Lister les comptes	GET	`/accounts`
F3	Consulter un compte	GET	`/accounts/{id}`
F4	Effectuer un dépôt	POST	`/accounts/{id}/deposit`
F5	Effectuer un retrait	POST	`/accounts/{id}/withdraw`
F+	Historique des transactions	GET	`/accounts/{id}/transactions`

3.3 Règles de gestion

1. Un compte possède un identifiant unique (UUID généré automatiquement)
2. Un compte possède un numéro de compte unique au format `ACC-AAAAMMJJ-XXXXXXXX`
3. Le solde initial doit être supérieur ou égal à zéro
4. Un dépôt doit obligatoirement avoir un montant strictement positif
5. Un retrait doit obligatoirement avoir un montant strictement positif
6. Un retrait est refusé si le solde du compte est insuffisant
7. Chaque opération modifie immédiatement et définitivement le solde
8. Chaque opération est enregistrée dans l'historique avec sa date et le solde après

4 Spécifications non fonctionnelles

Critère	Exigence	Comment c'est respecté
Performance	Réponses rapides	FastAPI + Uvicorn (ASGI asynchrone), pas de base de données distante
Fiabilité	Données persistées	Fichier `accounts.json` protégé par un verrou `threading.Lock`
Sécurité	Montants invalides bloqués	Pydantic valide `gt=0` et `ge=0` avant tout traitement
Maintenabilité	Code modulaire	3 fichiers avec rôles clairs (`main`, `models`, `storage`)
Évolutivité	Extensible	Architecture prête pour authentification JWT, virements, base SQL
Disponibilité	API accessible	Fonctionne tant que le serveur Uvicorn est actif

5 Implémentation détaillée — fichier par fichier

5.1 models.py — Définition des structures de données

Ce fichier définit tous les **schémas de données** utilisés par l'API. Chaque classe hérite de `BaseModel` de Pydantic, ce qui permet la validation automatique des données entrantes.

5.1.1 AccountCreate — Données pour créer un compte

```python
class AccountCreate(BaseModel):
    full_name: str = Field(
        ...,              # champ obligatoire (les ... signifient "requis")
        min_length=3,     # au moins 3 caracteres
        max_length=120,
        description="Nom complet du titulaire du compte.",
        examples=["KOA ESSAMA PAULIN BRICE"],
    )
    phone_number: str = Field(
        ...,              # champ obligatoire
        min_length=6,
        max_length=20,
        description="Numero de telephone du client.",
        examples=["699001122"],
    )
    email: str | None = Field(
        default=None,     # optionnel : si absent, vaut None
        max_length=120,
        description="Adresse email du client.",
        examples=["paulin@example.com"],
    )
    initial_balance: float = Field(
        default=0,        # optionnel : defaut = 0
        ge=0,             # ge = "greater or equal" : interdit les negatifs
        description="Solde initial. Doit etre superieur ou egal a zero.",
        examples=[10000],
    )
```

Listing 2 – models.py — AccountCreate

> **Comment Pydantic fonctionne :** Quand une requête POST arrive avec un corps JSON, Pydantic lit automatiquement les champs, vérifie les contraintes (`min_length`, `ge`, etc.) et retourne une erreur HTTP 422 si les données sont invalides — sans que le développeur écrive une seule ligne de validation.

5.1.2 TransactionRequest — Données pour un dépôt ou retrait

```python
class TransactionRequest(BaseModel):
    amount: float = Field(
        ...,     # obligatoire
        gt=0,    # gt = "greater than" : strictement positif
        description="Montant de l'operation. Doit etre strictement positif.",
```

```
        examples=[5000],
    )
    description: str | None = Field(
        default=None,   # optionnel : libelle de l'operation
        max_length=150,
        description="Libelle ou motif de la transaction.",
        examples=["Depot agence"],
    )
```

Listing 3 – models.py — TransactionRequest

5.1.3 Autres modèles — Réponses de l'API

```
class AccountResponse(BaseModel):
    id: str                     # UUID du compte
    account_number: str         # ex: ACC-20260422-A1B2C3D4
    full_name: str
    phone_number: str
    email: str | None
    balance: float              # solde courant
    created_at: str             # date ISO 8601

class TransactionResponse(BaseModel):
    transaction_id: str
    transaction_type: Literal["deposit", "withdraw"]
    amount: float
    description: str | None
    balance_after: float        # solde apres l'operation
    created_at: str

class AccountDetails(AccountResponse):
    # Herite de AccountResponse et ajoute les transactions
    transactions: list[TransactionResponse] =
        Field(default_factory=list)

class ErrorResponse(BaseModel):
    detail: str                 # Message d'erreur retourne par l'API
```

Listing 4 – models.py — Modeles de reponse

5.2 storage.py — Persistance des données

Ce fichier gère toute la **lecture et écriture des données** dans le fichier JSON local.

5.2.1 Configuration et utilitaires

```
import json
from pathlib import Path
from threading import Lock     # protege les ecritures concurrentes
from uuid import uuid4
from datetime import datetime, UTC

```

```python
BASE_DIR = Path(__file__).resolve().parent.parent    # racine du projet
DATA_FILE = BASE_DIR / "data" / "accounts.json"     # chemin du fichier JSON
FILE_LOCK = Lock()     # verrou pour empecher 2 ecritures simultanees

def _now() -> str:
    # Retourne la date/heure actuelle au format ISO 8601
    # Ex: "2026-04-22T14:30:00Z"
    return datetime.now(UTC).isoformat().replace("+00:00", "Z")

def _generate_account_number() -> str:
    # Genere un numero de compte unique
    # Ex: "ACC-20260422-A1B2C3D4"
    date_part = datetime.now(UTC).strftime("%Y%m%d")
    random_part = uuid4().hex[:8].upper()    # 8 caracteres aleatoires
    return f"ACC-{date_part}-{random_part}"
```

Listing 5 – storage.py — Configuration

5.2.2 Initialisation et lecture/écriture

```python
def initialize_storage() -> None:
    # Cree le dossier data/ et le fichier accounts.json s'ils n'existent pas
    DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
    if not DATA_FILE.exists():
        DATA_FILE.write_text(
            json.dumps({"accounts": []}, indent=2),
            encoding="utf-8"
        )

def _read_data() -> dict:
    # Lit et desserialise le fichier JSON
    initialize_storage()     # s'assure que le fichier existe
    return json.loads(DATA_FILE.read_text(encoding="utf-8"))

def _write_data(data: dict) -> None:
    # Serialise et ecrit les donnees dans le fichier JSON
    DATA_FILE.write_text(json.dumps(data, indent=2), encoding="utf-8")
```

Listing 6 – storage.py — Lecture et ecriture

Structure du fichier `accounts.json` :
Le fichier contient un objet JSON avec une clé `"accounts"` dont la valeur est un tableau de tous les comptes. Chaque compte contient ses transactions intégrées directement.

```json
{
  "accounts": [
    {
      "id": "a3f1b2c4-...",
      "account_number": "ACC-20260422-A1B2C3D4",
      "full_name": "KOA ESSAMA PAULIN BRICE",
```

```
      "phone_number": "699001122",
      "email": "paulin@example.com",
      "balance": 8000.0,
      "created_at": "2026-04-22T10:00:00Z",
      "transactions": [
        {
          "transaction_id": "f1e2d3c4-...",
          "transaction_type": "deposit",
          "amount": 10000.0,
          "description": "Solde initial",
          "balance_after": 10000.0,
          "created_at": "2026-04-22T10:00:00Z"
        },
        {
          "transaction_id": "b5a6c7d8-...",
          "transaction_type": "withdraw",
          "amount": 2000.0,
          "description": "Retrait DAB",
          "balance_after": 8000.0,
          "created_at": "2026-04-22T11:30:00Z"
        }
      ]
    }
  ]
}
```

Listing 7 – Exemple de contenu de accounts.json

5.2.3 Création d'un compte

```python
def create_account(payload: dict) -> dict:
    with FILE_LOCK:     # verrou : une seule operation d'ecriture a la fois
        data = _read_data()

        account = {
            "id": str(uuid4()),                         # UUID unique
            "account_number": _generate_account_number(),
            "full_name": payload["full_name"],
            "phone_number": payload["phone_number"],
            "email": payload.get("email"),              # None si absent
            "balance": round(float(payload.get("initial_balance", 0)), 2),
            "created_at": _now(),
            "transactions": [],                         # liste vide au depart
        }

        # Si un solde initial est fourni, on cree une premiere transaction
        if account["balance"] > 0:
            account["transactions"].append({
                "transaction_id": str(uuid4()),
                "transaction_type": "deposit",
                "amount": account["balance"],
                "description": "Solde initial",
                "balance_after": account["balance"],
```

```
                "created_at": _now(),
            })

        data["accounts"].append(account)     # ajout dans la liste
        _write_data(data)                    # sauvegarde dans le fichier
        return account
```

Listing 8 – storage.py — create_account()

5.2.4 Application d'une transaction (dépôt ou retrait)

```
def apply_transaction(
    account_id: str,
    transaction_type: str,    # "deposit" ou "withdraw"
    amount: float,
    description: str | None
) -> dict | None:

    with FILE_LOCK:      # verrou pour l'ecriture atomique
        data = _read_data()

        for account in data["accounts"]:
            if account["id"] != account_id:
                continue     # passe au compte suivant si ce n'est pas le bon

            current_balance = float(account["balance"])
            amount = round(float(amount), 2)

            # Verification du solde avant un retrait
            if transaction_type == "withdraw" and amount > current_balance:
                raise ValueError("Solde insuffisant pour effectuer ce retrait.")

            # Calcul du nouveau solde
            if transaction_type == "deposit":
                new_balance = round(current_balance + amount, 2)
            else:
                new_balance = round(current_balance - amount, 2)

            # Creation de la transaction
            transaction = {
                "transaction_id": str(uuid4()),
                "transaction_type": transaction_type,
                "amount": amount,
                "description": description,
                "balance_after": new_balance,
                "created_at": _now(),
            }

            account["balance"] = new_balance             # mise a jour du solde
            account["transactions"].append(transaction) # ajout dans l'historique
            _write_data(data)                            # sauvegarde
```

```
            return account

        return None      # compte non trouve
```

Listing 9 – storage.py — apply_transaction()

5.3 main.py — Les routes de l'API

Ce fichier est le cœur de l'API. Il définit toutes les **routes HTTP** et orchestre les appels aux autres modules.

5.3.1 Initialisation de FastAPI

```
from fastapi import FastAPI, HTTPException, status
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.staticfiles import StaticFiles

app = FastAPI(
    title="API de transaction bancaire",
    description="...",    # description longue visible dans Swagger
    version="1.0.0",
    docs_url=None,        # on desactive la route /docs par defaut
    redoc_url=None,       # pour utiliser notre propre Swagger UI local
    openapi_tags=[
        {"name": "General", "description": "Informations generales."},
        {"name": "Accounts", "description": "Gestion des comptes."},
        {"name": "Transactions", "description": "Depots et retraits."},
    ],
)

# Sert les fichiers du dossier app/static/ a l'URL /static/
app.mount("/static", StaticFiles(directory="app/static"), name="static")

@app.on_event("startup")
def startup() -> None:
    initialize_storage()     # cree le fichier JSON au demarrage si besoin
```

Listing 10 – main.py — Initialisation de l'application

5.3.2 Fonction 1 — Créer un compte

```
@app.post(
    "/accounts",
    response_model=AccountResponse,
    status_code=status.HTTP_201_CREATED,    # HTTP 201 Created (pas 200)
    tags=["Accounts"],
    summary="Creer un compte bancaire",
```

```python
)
def create_bank_account(payload: AccountCreate) -> dict:
    # FastAPI injecte automatiquement payload apres validation Pydantic
    account = create_account(payload.model_dump())   # cree et persiste
    # On retourne le compte sans la liste de transactions
    return {key: value for key, value in account.items() if key != "transactions"}
```

Listing 11 – main.py — POST /accounts

Ce qui se passe étape par étape :

1. Le client envoie un POST avec un corps JSON
2. FastAPI lit le corps et le passe à Pydantic (`AccountCreate`)
3. Pydantic vérifie les contraintes (champs obligatoires, longueurs, `ge=0`)
4. Si invalide → FastAPI retourne automatiquement HTTP 422 avec le détail
5. Si valide → `create_account()` est appelé, le compte est créé et sauvegardé
6. La réponse HTTP 201 est retournée avec les données du nouveau compte

5.3.3 Fonction 2 — Lister les comptes

```python
@app.get(
    "/accounts",
    response_model=list[AccountResponse],   # liste de comptes en reponse
    tags=["Accounts"],
    summary="Lister tous les comptes",
)
def get_accounts() -> list[dict]:
    accounts = list_accounts()    # lit tous les comptes depuis le fichier JSON
    # Retourne chaque compte sans ses transactions (pour alleges la reponse)
    return [
        {key: value for key, value in account.items() if key != "transactions"}
        for account in accounts
    ]
```

Listing 12 – main.py — GET /accounts

5.3.4 Fonction 3 — Consulter un compte

```python
@app.get(
    "/accounts/{account_id}",   # {account_id} est un parametre de chemin
    response_model=AccountDetails,  # inclut les transactions
    tags=["Accounts"],
    summary="Consulter un compte",
    responses={404: {"model": ErrorResponse}},
)
def get_account_by_id(account_id: str) -> dict:
```

```
    account = get_account(account_id)   # cherche dans le fichier JSON
    if not account:
        # Lance une exception HTTP 404 si le compte n'existe pas
        raise HTTPException(status_code=404, detail="Compte introuvable.")
    return account   # retourne le compte avec ses transactions
```

Listing 13 – main.py — GET /accounts/{account_id}

5.3.5 Fonction 4 — Effectuer un dépôt

```
@app.post(
    "/accounts/{account_id}/deposit",
    response_model=AccountDetails,
    tags=["Transactions"],
    summary="Effectuer un depot",
    responses={
        404: {"model": ErrorResponse, "description": "Compte introuvable."},
    },
)
def deposit_money(account_id: str, payload: TransactionRequest) -> dict:
    # Appelle apply_transaction avec type "deposit"
    updated_account = apply_transaction(
        account_id=account_id,
        transaction_type="deposit",
        amount=payload.amount,
        description=payload.description,
    )
    if not updated_account:
        # apply_transaction retourne None si le compte n'existe pas
        raise HTTPException(status_code=404, detail="Compte introuvable.")
    return updated_account   # retourne le compte mis a jour
```

Listing 14 – main.py — POST /accounts/{account_id}/deposit

5.3.6 Fonction 5 — Effectuer un retrait

```
@app.post(
    "/accounts/{account_id}/withdraw",
    response_model=AccountDetails,
    tags=["Transactions"],
    summary="Effectuer un retrait",
    responses={
        400: {"model": ErrorResponse, "description": "Solde insuffisant."},
        404: {"model": ErrorResponse, "description": "Compte introuvable."},
    },
)
def withdraw_money(account_id: str, payload: TransactionRequest) -> dict:
```

```python
    try:
        updated_account = apply_transaction(
            account_id=account_id,
            transaction_type="withdraw",
            amount=payload.amount,
            description=payload.description,
        )
    except ValueError as error:
        # apply_transaction leve ValueError si solde insuffisant
        # On la convertit en erreur HTTP 400
        raise HTTPException(status_code=400, detail=str(error)) from error

    if not updated_account:
        raise HTTPException(status_code=404, detail="Compte introuvable.")
    return updated_account
```

Listing 15 – main.py — POST /accounts/{account_id}/withdraw

6 Cas de test

6.1 Tableau complet des 23 cas de test

ID	Fonction	Description	Résultat attendu	Code HTTP
CT-01	Créer compte	Tous les champs fournis	Compte créé avec ID et numéro	201
CT-02	Créer compte	Sans email ni solde initial	Compte créé, balance = 0	201
CT-03	Créer compte	`full_name` absent	Erreur de validation	422
CT-04	Créer compte	`phone_number` absent	Erreur de validation	422
CT-05	Créer compte	Solde initial négatif	Erreur de validation	422
CT-06	Lister comptes	Aucun compte créé	Tableau vide `[]`	200
CT-07	Lister comptes	Comptes existants	Liste des comptes	200
CT-08	Consulter compte	ID valide	Détails + transactions	200
CT-09	Consulter compte	ID invalide	Compte introuvable	404
CT-10	Dépôt	Montant valide	Solde augmente	200
CT-11	Dépôt	Avec description	Libellé visible	200

ID	Fonction	Description	Résultat attendu	Code HTTP
CT-12	Dépôt	Montant = 0	Erreur validation	422
CT-13	Dépôt	Montant négatif	Erreur validation	422
CT-14	Dépôt	Compte inexistant	Compte introuvable	404
CT-15	Retrait	Solde suffisant	Solde diminue	200
CT-16	Retrait	Montant = solde	Solde = 0	200
CT-17	Retrait	Solde insuffisant	Solde insuffisant	400
CT-18	Retrait	Montant = 0	Erreur validation	422
CT-19	Retrait	Montant négatif	Erreur validation	422
CT-20	Retrait	Compte inexistant	Compte introuvable	404
CT-21	Transactions	Compte sans opération	Tableau vide	200
CT-22	Transactions	Avec opérations	Liste chronologique	200
CT-23	Transactions	Compte inexistant	Compte introuvable	404

6.2 Test manuel détaillé — Scénario complet

Le test manuel se fait directement dans **Swagger UI** accessible à `http://localhost:8000/docs`.

6.2.1 Étape 1 — Vérifier la liste vide (CT-06)

```
GET http://localhost:8000/accounts
```

Listing 16 – Appel GET /accounts

> **Résultat attendu :** HTTP 200 avec le corps `[]`
> Cela confirme que la persistance est initialisée et que le système fonctionne.

6.2.2 Étape 2 — Créer un compte (CT-01)

```
POST http://localhost:8000/accounts
Content-Type: application/json

{
  "full_name": "KOA ESSAMA PAULIN BRICE",
  "phone_number": "699001122",
```

```
  "email": "paulin@example.com",
  "initial_balance": 10000
}
```

Listing 17 – Appel POST /accounts

```
{
  "id": "a3f1b2c4-d5e6-7890-abcd-ef1234567890",
  "account_number": "ACC-20260422-A1B2C3D4",
  "full_name": "KOA ESSAMA PAULIN BRICE",
  "phone_number": "699001122",
  "email": "paulin@example.com",
  "balance": 10000.0,
  "created_at": "2026-04-22T10:00:00Z"
}
```

Listing 18 – Reponse attendue — HTTP 201

À noter : Copier l'`id` retourné pour l'utiliser dans les étapes suivantes.

6.2.3 Étape 3 — Tester la validation (CT-03, CT-04, CT-05)

```
POST http://localhost:8000/accounts
Content-Type: application/json

{
  "full_name": "TEST",
  "phone_number": "600000001",
  "initial_balance": -500
}
```

Listing 19 – Test CT-05 : solde negatif

Résultat attendu : HTTP 422 avec un message d'erreur Pydantic indiquant que `initial_balance` doit être ≥ 0.

6.2.4 Étape 4 — Effectuer un dépôt (CT-10)

```
POST http://localhost:8000/accounts/a3f1b2c4-.../deposit
Content-Type: application/json

{
  "amount": 5000,
  "description": "Depot agence"
}
```

Listing 20 – Appel POST /accounts/{id}/deposit

```
{
  "id": "a3f1b2c4-...",
  "balance": 15000.0,
  "transactions": [
```

```
    {
      "transaction_type": "deposit",
      "amount": 10000.0,
      "description": "Solde initial",
      "balance_after": 10000.0
    },
    {
      "transaction_type": "deposit",
      "amount": 5000.0,
      "description": "Depot agence",
      "balance_after": 15000.0
    }
  ]
}
```

Listing 21 – Reponse attendue — HTTP 200, balance = 15000

6.2.5 Étape 5 — Effectuer un retrait valide (CT-15)

```
POST http://localhost:8000/accounts/a3f1b2c4-.../withdraw
Content-Type: application/json

{
  "amount": 2000,
  "description": "Retrait DAB"
}
```

Listing 22 – Appel POST /accounts/{id}/withdraw

Résultat attendu : HTTP 200, balance = 13000 (15000 - 2000).

6.2.6 Étape 6 — Tester le solde insuffisant (CT-17)

```
POST http://localhost:8000/accounts/a3f1b2c4-.../withdraw
Content-Type: application/json

{
  "amount": 50000,
  "description": "Test solde insuffisant"
}
```

Listing 23 – Test CT-17 : retrait impossible

Résultat attendu : HTTP 400
Corps : `{"detail": "Solde insuffisant pour effectuer ce retrait."}`

6.2.7 Étape 7 — Consulter l'historique (CT-22)

```
GET http://localhost:8000/accounts/a3f1b2c4-.../transactions
```

Listing 24 – Appel GET /accounts/{id}/transactions

Résultat attendu : HTTP 200 avec 3 transactions dans l'ordre chronologique :
1. Dépôt initial (10 000)
2. Dépôt agence (5 000) → balance 15 000
3. Retrait DAB (2 000) → balance 13 000

6.2.8 Étape 8 — Tester un ID inexistant (CT-09)

```
GET http://localhost:8000/accounts
    /00000000-0000-0000-0000-000000000000
```

Listing 25 – Test CT-09 : compte inexistant

Résultat attendu : HTTP 404
Corps : `{"detail": "Compte introuvable."}`

7 Documentation Swagger

7.1 Accès à l'interface

L'interface Swagger UI est accessible à l'adresse `http://localhost:8000/docs` une fois le serveur lancé. Elle permet de tester toutes les fonctions de l'API directement depuis le navigateur, sans outil externe.

7.2 Exemples interactifs dans Swagger

L'API intègre des **exemples nommés** dans Swagger pour chaque endpoint de création et de transaction. Ces exemples apparaissent dans un menu déroulant *« Examples »* permettant de basculer entre les cas de test.

Endpoint	Exemples disponibles
`POST /accounts`	CT-01, CT-02, CT-03, CT-04, CT-05
`POST /{id}/deposit`	CT-10, CT-11, CT-12, CT-13
`POST /{id}/withdraw`	CT-15, CT-16, CT-17, CT-18, CT-19

7.3 Comment utiliser Swagger pour le test manuel

1. Lancer l'API : `uvicorn app.main:app --reload`
2. Ouvrir `http://localhost:8000/docs` dans le navigateur
3. Cliquer sur un endpoint (ex : *POST /accounts*)
4. Cliquer sur le bouton « **Try it out** »
5. Sélectionner un exemple dans le menu déroulant « **Examples** »
6. Cliquer sur « **Execute** »
7. Observer le code HTTP et le corps de la réponse dans la section *Responses*

8 Lancement et déploiement

8.1 Lancement en local

```
# 1. Installer les dependances
pip install -r requirements.txt

# 2. Lancer le serveur (mode developpement avec rechargement
    automatique)
uvicorn app.main:app --reload

# 3. L'API est disponible sur :
#    http://localhost:8000/
#    http://localhost:8000/docs  (Swagger UI)
```

Listing 26 – Lancement de l'API en local

8.2 Contenu de requirements.txt

```
fastapi
uvicorn[standard]
```

Listing 27 – requirements.txt

8.3 Déploiement sur Render

Le fichier `render.yaml` à la racine du projet configure le déploiement automatique sur la plateforme cloud **Render.com**. À chaque push sur la branche `main` de GitHub, le service est automatiquement mis à jour.

9 Conclusion

Ce projet implémente une **API REST complète** pour la gestion de comptes bancaires, respectant toutes les exigences fonctionnelles et non fonctionnelles définies dans le cahier des charges.

9.0 Points clés de l'implémentation

- **Séparation des responsabilités** : 3 modules distincts avec des rôles clairs
- **Validation automatique** : Pydantic assure la validité de toutes les données entrantes
- **Persistance fiable** : fichier JSON protégé par un verrou thread-safe
- **Documentation interactive** : 23 cas de test directement exécutables dans Swagger
- **Codes HTTP corrects** : 201 (création), 200 (succès), 400 (logique métier), 404 (non trouvé), 422 (validation)

9.0 Ce que ce projet ne fait pas encore (limites)

— Pas d'authentification (JWT ou sessions)
— Pas de base de données relationnelle (SQLite, PostgreSQL)
— Pas de virement entre comptes
— Pas de séparation des rôles client/administrateur

Ces éléments constituent des évolutions naturelles pour une version future du système.

KOA ESSAMA PAULIN BRICE — Matricule 25I2255 — Licence 3 ICT
Université de Yaoundé I — Avril 2026